

O'REILLY®



图灵程序设计丛书

流畅的 Python

Fluent Python

PSF研究员、知名PyCon演讲者心血之作
全面深入，对Python语言关键特性剖析到位

[巴西] Luciano Ramalho 著
安道 吴珂 译



中国工信出版集团



人民邮电出版社
POSTS & TELECOM PRESS

see more please visit: <https://homeofpdf.com>

版权信息

书名：流畅的Python

作者：[巴西] Luciano Ramalho

译者：安道 吴珂

ISBN：978-7-115-45415-7

本书由北京图灵文化发展有限公司发行数字版。版权所有，侵权必究。

您购买的图灵电子书仅供您个人使用，未经授权，不得以任何方式复制和传播本书内容。

我们愿意相信读者具有这样的良知和觉悟，与我们共同保护知识产权。

如果购买者有侵权行为，我们可能对该用户实施包括但不限于关闭该帐号等维权措施，并可能追究法律责任。

图灵社区会员 boat (727442652@qq.com) 专享 尊重版权

版权声明

O'Reilly Media, Inc. 介绍

业界评论

前言

目标读者

非目标读者

本书的结构

以实践为基础

硬件

杂谈：个人的一点看法

Python术语表

Python版本表

排版约定

使用代码示例

Safari® Books Online

联系我们

致谢

电子书

第一部分 序幕

第1章 Python 数据模型

1.1 一掬Python风格的纸牌

1.2 如何使用特殊方法

1.2.1 模拟数值类型

1.2.2 字符串表示形式

1.2.3 算术运算符

1.2.4 自定义的布尔值

1.3 特殊方法一览

1.4 为什么len不是普通方法

1.5 本章小结

1.6 延伸阅读

第二部分 数据结构

第2章 序列构成的数组

2.1 内置序列类型概览

2.2 列表推导和生成器表达式

2.2.1 列表推导和可读性

2.2.2 列表推导同filter和map的比较

2.2.3 笛卡儿积

2.2.4 生成器表达式

2.3 元组不仅仅是不可变的列表

2.3.1 元组和记录

2.3.2 元组拆包

2.3.3 嵌套元组拆包

2.3.4 具名元组

2.3.5 作为不可变列表的元组

2.4 切片

2.4.1 为什么切片和区间会忽略最后一个元素

2.4.2 对对象进行切片

2.4.3 多维切片和省略

2.4.4 给切片赋值

2.5 对序列使用+和*

建立由列表组成的列表

2.6 序列的增量赋值

一个关于+=的谜题

2.7 list.sort方法和内置函数sorted

2.8 用bisect来管理已排序的序列

2.8.1 用bisect来搜索

- 2.8.2 用bisect.insort插入新元素
- 2.9 当列表不是首选时
 - 2.9.1 数组
 - 2.9.2 内存视图
 - 2.9.3 NumPy和SciPy
 - 2.9.4 双向队列和其他形式的队列
- 2.10 本章小结
- 2.11 延伸阅读
- 第3章 字典和集合
 - 3.1 泛映射类型
 - 3.2 字典推导
 - 3.3 常见的映射方法
 - 用setdefault处理找不到的键
 - 3.4 映射的弹性键查询
 - 3.4.1 defaultdict: 处理找不到的键的一个选择
 - 3.4.2 特殊方法__missing__
 - 3.5 字典的变种
 - 3.6 子类化UserDict
 - 3.7 不可变映射类型
 - 3.8 集合论
 - 3.8.1 集合字面量
 - 3.8.2 集合推导
 - 3.8.3 集合的操作
 - 3.9 dict和set的背后
 - 3.9.1 一个关于效率的实验
 - 3.9.2 字典中的散列表
 - 3.9.3 dict的实现及其导致的结果
 - 3.9.4 set的实现以及导致的结果

- 3.10 本章小结
- 3.11 延伸阅读
- 第4章 文本和字节序列
 - 4.1 字符问题
 - 4.2 字节概要
 - 结构体和内存视图
 - 4.3 基本的编解码器
 - 4.4 了解编解码问题
 - 4.4.1 处理UnicodeEncodeError
 - 4.4.2 处理UnicodeDecodeError
 - 4.4.3 使用预期之外的编码加载模块时抛出的SyntaxError
 - 4.4.4 如何找出字节序列的编码
 - 4.4.5 BOM: 有用的鬼符
 - 4.5 处理文本文件
 - 编码默认值: 一团糟
 - 4.6 为了正确比较而规范化Unicode字符串
 - 4.6.1 大小写折叠
 - 4.6.2 规范化文本匹配实用函数
 - 4.6.3 极端“规范化”: 去掉变音符号
 - 4.7 Unicode文本排序
 - 使用Unicode排序算法排序
 - 4.8 Unicode数据库
 - 4.9 支持字符串和字节序列的双模式API
 - 4.9.1 正则表达式中的字符串和字节序列
 - 4.9.2 os函数中的字符串和字节序列
 - 4.10 本章小结
 - 4.11 延伸阅读
- 第三部分 把函数视作对象

第 5 章 一等函数

5.1 把函数视作对象

5.2 高阶函数

`map`、`filter`和`reduce`的现代替代品

5.3 匿名函数

5.4 可调用对象

5.5 用户定义的可调用类型

5.6 函数内省

5.7 从定位参数到仅限关键字参数

5.8 获取关于参数的信息

5.9 函数注解

5.10 支持函数式编程的包

5.10.1 `operator`模块

5.10.2 使用`functools.partial`冻结参数

5.11 本章小结

5.12 延伸阅读

第 6 章 使用一等函数实现设计模式

6.1 案例分析：重构“策略”模式

6.1.1 经典的“策略”模式

6.1.2 使用函数实现“策略”模式

6.1.3 选择最佳策略：简单的方式

6.1.4 找出模块中的全部策略

6.2 “命令”模式

6.3 本章小结

6.4 延伸阅读

第 7 章 函数装饰器和闭包

7.1 装饰器基础知识

7.2 Python何时执行装饰器

- 7.3 使用装饰器改进“策略”模式
- 7.4 变量作用域规则
- 7.5 闭包
- 7.6 `nonlocal`声明
- 7.7 实现一个简单的装饰器
 - 工作原理
- 7.8 标准库中的装饰器
 - 7.8.1 使用`functools.lru_cache`做备忘
 - 7.8.2 单分派泛函数
- 7.9 叠放装饰器
- 7.10 参数化装饰器
 - 7.10.1 一个参数化的注册装饰器
 - 7.10.2 参数化`clock`装饰器
- 7.11 本章小结
- 7.12 延伸阅读

第四部分 面向对象惯用法

第8章 对象引用、可变性和垃圾回收

- 8.1 变量不是盒子
- 8.2 标识、相等性和别名
 - 8.2.1 在`==`和`is`之间选择
 - 8.2.2 元组的相对不可变性
- 8.3 默认做浅复制
 - 为任意对象做深复制和浅复制
- 8.4 函数的参数作为引用时
 - 8.4.1 不要使用可变类型作为参数的默认值
 - 8.4.2 防御可变参数
- 8.5 `del`和垃圾回收
- 8.6 弱引用

- 8.6.1 WeakValueDictionary简介
 - 8.6.2 弱引用的局限
- 8.7 Python对不可变类型施加的把戏
- 8.8 本章小结
- 8.9 延伸阅读
- 第9章 符合Python风格的对象
 - 9.1 对象表示形式
 - 9.2 再谈向量类
 - 9.3 备选构造方法
 - 9.4 classmethod与staticmethod
 - 9.5 格式化显示
 - 9.6 可散列的Vector2d
 - 9.7 Python的私有属性和“受保护的”属性
 - 9.8 使用 __slots__ 类属性节省空间
 - __slots__ 的问题
 - 9.9 覆盖类属性
 - 9.10 本章小结
 - 9.11 延伸阅读
- 第10章 序列的修改、散列和切片
 - 10.1 Vector类：用户定义的序列类型
 - 10.2 Vector类第1版：与Vector2d类兼容
 - 10.3 协议和鸭子类型
 - 10.4 Vector类第2版：可切片的序列
 - 10.4.1 切片原理
 - 10.4.2 能处理切片的__getitem__方法
 - 10.5 Vector类第3版：动态存取属性
 - 10.6 Vector类第4版：散列和快速等值测试
 - 10.7 Vector类第5版：格式化